Procedural Planet Generator PRO - Documentation

Overview

Procedural Planet Generator PRO is a powerful Unity asset designed to create procedurally generated planets with customizable terrain, biomes, oceans, and vegetation. Inspired by games like No Man's Sky, this asset provides a robust system for generating spherical worlds with dynamic Level of Detail (LOD) and a first-person controller tailored for planetary exploration. It is ideal for developers creating sci-fi games, space exploration simulators, or any project requiring detailed, procedurally generated planetary environments.

Features

Procedural Planet Generation: Generate spherical planets with varied terrain using layered noise functions (Perlin-based FBM).
Dynamic LOD System: Optimizes performance with multiple LOD levels for terrain chunks, adjustable based on distance from the player.
Biome System: Supports diverse biomes with customizable colors for deep water, shallow water, sand, grass, forest, rock, and snow.
Ocean Generation: Optional procedurally generated oceans with customizable resolution and depth.
Vegetation: Procedural tree spawning with customizable density and slope constraints.
Planetary First-Person Controller: A dedicated controller for planetary movement with gravity aligned to the planet's surface, supporting walking, sprinting, and jumping.
Domain Warping: Enhances terrain realism with organic distortions.
Performance Optimization: Asynchronous chunk generation, configurable chunk updates, and optional mesh colliders for improved performance.
Customizable Parameters: Extensive settings for terrain, noise, biomes, LOD, and more, accessible via the Unity Inspector.

Installation

Import the Asset:
Import the Procedural Planet Generator PRO package into your Unity project via the Unity Asset Store or by importing the .unitypackage file.

Set Up the Scene:
Add the PlanetGenerator script to an empty GameObject to serve as the planet's root.
Attach the PlanetaryFirstPersonController script to your player GameObject, ensuring it has a Rigidbody and CapsuleCollider.

Configure Materials:
Assign a planet material (preferably with a shader supporting vertex colors) to the Planet Material field in the PlanetGenerator component.
Optionally, assign an ocean material for water rendering.

Set Up the Camera:
Assign the main camera to the Player Camera field in the PlanetaryFirstPersonController component.

Test the Scene:
Press Play to generate the planet and test the first-person controller.

Components

1. PlanetGenerator

The core component responsible for generating the planet's terrain, oceans, and vegetation.

Key Settings

Planet:
Radius: The base radius of the planet (default: 1000 units).

Planet Material: Material for rendering the planet's surface.

Ocean:
Generate Ocean: Enable/disable ocean generation.
Ocean Material: Material for the ocean mesh.
Ocean Level: Height of the ocean relative to the planet's radius.
Ocean Resolution: Number of vertices per face for the ocean mesh (4-128).

Chunks and LOD:
Max Verts Per Chunk: Maximum vertices per chunk edge for the highest LOD.
Chunks Per Face: Number of chunks per planet face (1-64).
LOD Levels: Number of LOD levels (1-4).
LOD Distances: Distance thresholds for each LOD level and unloading.

Terrain:
Height Multiplier: Scales the terrain height.
Base Noise Scale, Base Octaves, Base Persistence, Base Lacunarity: Control the base terrain noise.
Mountain Noise Scale, Mountain Octaves, Mountain Strength, Mountain Sharpness: Define mountain features.
Mega Noise Scale, Mega Octaves, Mega Strength, Mega Rarity, Mega Sharpness: Control large, rare peaks.
Valley Noise Scale, Valley Octaves, Valley Strength: Define valley features.
Detail Noise Scale, Detail Strength: Add fine terrain details.
Ocean Threshold, Ocean Depth: Control ocean placement and depth.
Warp Scale, Warp Strength: Apply domain warping for organic terrain.

Biome Colors:
Colors for deep water, shallow water, sand, grass, forest, rock, and snow.
Snow Height Start: Altitude where snow appears.

Performance:
Max Chunks Per Frame: Limits chunk generation per frame.
Update Interval: Frequency of chunk updates.
Use Colliders: Enable/disable mesh colliders for terrain.
Collider Distance: Distance within which colliders are enabled.

Trees:
Tree Prefab: Custom tree prefab or default simple tree.
Tree Probability: Likelihood of tree spawning.
Max Tree Slope: Maximum terrain slope for tree placement.

UI:
Water Panel: UI panel shown when the player is underwater.
Loading Panel: UI panel shown during initial planet generation.
Extra Object To Hide: Additional object to hide post-loading.

Scalable Objects:
First Object, Second Object: Objects scaled based on planet radius (e.g., atmosphere, skybox).

Custom Spawn:
Enable Surface Spawn: Spawn a custom prefab on the planet's surface.
Spawn Prefab: Prefab to spawn at the initial position.

Usage

Attach the PlanetGenerator script to a GameObject.
Configure parameters in the Inspector to customize the planet's appearance and performance.
Ensure a valid material is assigned to avoid rendering issues.
For multi-planet scenes, create multiple GameObjects with PlanetGenerator components.

2. PlanetaryFirstPersonController
A first-person controller designed for planetary surfaces, aligning gravity and movement to the planet's curvature.
Key Settings

References:
Planet: The planet's transform (auto-detected if Auto Detect Planets is enabled).
Player Camera: The camera for first-person view.
Ground Layers: Layers for ground detection.

Movement:
Move Speed: Base movement speed.
Sprint Multiplier: Speed multiplier when sprinting.
Jump Height: Height of the player's jump.
Air Control: Movement control while airborne (0-1).
Ground Stick Speed: Downward force to keep the player grounded.

Mouse Look:
Mouse Sensitivity: Sensitivity for mouse look.
Mouse Smoothing: Smoothing factor for mouse input.
Max Look Angle: Maximum pitch angle for camera.
Invert Y: Invert vertical mouse input.

Gravity:
Use Inverse Square: Apply inverse-square gravity based on distance.
Gravity At Surface: Gravity strength at the planet's surface.
Planet Radius: Radius of the planet (synced with PlanetGenerator if auto-detected).
Ground Check Distance: Distance for ground detection.

Auto-detect Planets:
Auto Detect Planets: Automatically find the nearest planet.
Gravity Range Multiplier: Multiplier for planet influence range.
Planet List Refresh Interval: Frequency of planet list updates.

Rotation / Smoothing:
Align Speed: Speed of aligning the player to the surface normal.

Spawn / Safety:
Spawn Clearance: Offset to resolve spawn inside geometry.
Spawn Search Distance: Maximum distance to find a valid spawn point.

PC Controls:
Enable PC Controls: Enable keyboard/mouse input.

Usage

Attach the PlanetaryFirstPersonController script to the player GameObject.
Ensure a Rigidbody and CapsuleCollider are present.
Assign the main camera to the Player Camera field.
Enable Auto Detect Planets for multi-planet scenes or manually assign a planet's transform.
Adjust movement and gravity settings for desired player behavior.

Setup Guide

Create a Planet:
Add an empty GameObject to the scene.
Attach the PlanetGenerator component.
Assign a planet material (e.g., Standard shader with vertex color support).
Configure terrain, biome, and LOD settings as needed.

Set Up the Player:
Create a player GameObject with a Rigidbody and CapsuleCollider.
Attach the PlanetaryFirstPersonController component.
Assign the main camera to the Player Camera field.
Set the Planet field to the planet GameObject or enable Auto Detect Planets.

Add Atmosphere (Optional):
Assign scalable objects (e.g., atmosphere spheres) to First Object and Second Object in PlanetGenerator.

Test and Optimize:
Run the scene to generate the planet and test player movement.
Adjust LOD distances, chunk settings, and performance parameters for optimal frame rate.

Customize:
Modify biome colors, noise parameters, and tree settings to create unique planets.
Use custom tree prefabs or ocean materials for specific aesthetics.

Performance Tips

LOD Configuration: Reduce Max Verts Per Chunk and Chunks Per Face for lower-end hardware.
Chunk Updates: Increase Update Interval or reduce Max Chunks Per Frame to spread out chunk generation.
Colliders: Disable Use Colliders or increase Collider Distance to reduce physics overhead.
Ocean Resolution: Lower Ocean Resolution for faster ocean rendering.
Tree Density: Decrease Tree Probability to reduce vegetation overhead.

Known Limitations

Shader Compatibility: The planet material must support vertex colors for biome coloring. Use the Standard shader or a custom shader with vertex color support.
Multi-Planet Performance: Auto-detection of multiple planets may impact performance in scenes with many planets. Adjust Planet List Refresh Interval to balance accuracy and performance.
Tree Prefabs: Custom tree prefabs must be lightweight to avoid performance issues.
Physics: The controller assumes a spherical planet for gravity calculations. Non-spherical objects may cause unexpected behavior.

Troubleshooting

Planet Not Generating: Ensure a valid material is assigned to Planet Material. Check the console for warnings about missing materials or invalid settings.
Player Falling Through Planet: Verify that Ground Layers includes the planet's terrain layers. Increase Ground Check Distance if needed.
Performance Issues: Reduce Chunks Per Face, lower Max Verts Per Chunk, or increase LOD Distances.
No Ocean Visible: Ensure Generate Ocean is enabled and an ocean material is assigned.
Controller Not Detecting Planet: Enable Auto Detect Planets or manually assign the planet's transform.

Example Scene

An example scene is included in the asset package, demonstrating a fully configured planet with a first-person controller. To use it:

Open the example scene from the project folder.
Ensure the main camera is assigned to the player's Player Camera field.
Play the scene to explore the planet.
Modify the PlanetGenerator settings to experiment with different terrain and biome configurations.